

DEVOPS MASTER COURSE

Volume 6B

AWS Core Infrastructure

Chapters 11-20

EC2 FLEETS | VPC | DNS | LOAD BALANCING | SCALING | STORAGE

Advanced design, operations, troubleshooting, labs, and interviews

2026 Edition

About This Part

Volume 6B advances from AWS foundations into core infrastructure engineering. It focuses on the decisions that make systems reproducible, resilient, secure, observable, recoverable, and cost-aware: instance fleet standards, fault-domain design, controlled traffic paths, health-aware routing, elastic capacity, storage performance, and verified recovery.

Scope and Safety

- Use a sandbox account or organizational unit.
- Create budgets before NAT gateways, load balancers, public IPv4 addresses, provisioned file systems, or cross-Region copies.
- Use Session Manager or private access rather than public SSH/RDP.
- Require IMDSv2, encrypt data, and never place credentials in AMIs, user data, source code, or logs.
- Delete chargeable lab resources after validation.
- Verify current AWS quotas, regional availability, features, and pricing before production use.

How to Use the Labs

- Read design and failure modes first.
- Tag every resource with owner, environment, application, cost center, and expiry.
- Record expected behavior before tests.
- Validate logs, metrics, routing, security, encryption, health, and cleanup.
- Repeat important tasks with CLI or Infrastructure as Code.

Table of Contents

Chapter 11 Advanced EC2 Administration

Chapter 12 Advanced Amazon VPC

Chapter 13 Security Groups, Network ACLs, and Segmentation

Chapter 14 Amazon Route 53 and DNS Engineering

Chapter 15 Elastic Load Balancing Deep Dive

Chapter 16 EC2 Auto Scaling Engineering

Chapter 17 Amazon EBS Deep Dive

Chapter 18 Amazon EFS and Amazon FSx

Chapter 19 Amazon S3 Masterclass

Chapter 20 Enterprise Storage Architecture, Backup, and Disaster Recovery

Appendices CLI Reference, Troubleshooting Matrix, Production Readiness Checklist, Glossary, and Sources

Navigation: Heading styles allow Word or LibreOffice to generate a field-based table of contents.

Advanced EC2 Administration

Versioned fleets, secure bootstrapping, placement, capacity, and lifecycle operations

VOLUME 6B | CHAPTER 11

Learning Objectives

- Use launch templates as versioned instance specifications.
- Require IMDSv2 and use instance profiles instead of stored credentials.
- Select placement, capacity, hibernation, and purchase options correctly.
- Operate immutable fleets and troubleshoot launch or readiness failures.

11.1 From servers to replaceable fleets

A manually configured instance is an operational exception: it is difficult to reproduce, audit, patch, and recover. Production teams define EC2 capacity through approved AMIs, launch templates, instance roles, network policy, observability, health checks, and replacement rules. Persistent state is externalized so unhealthy or noncompliant instances can be replaced instead of repaired indefinitely.

The fleet definition belongs in version control and a delivery pipeline. A change is tested as a new image or launch-template version, deployed to a limited scope, observed, and then promoted. This separates an approved release from the latest change and creates a practical rollback path.

11.2 Launch templates and image strategy

Launch templates store launch parameters including AMI, type, storage, security groups, instance profile, metadata options, user data, monitoring, placement, and tags. Versions support audit and rollback. Production Auto Scaling groups should consume an explicitly approved version or a controlled promotion mechanism.

Most mature platforms use a hybrid image strategy. Stable operating-system packages, security agents, runtimes, and baseline configuration are baked into a golden AMI. Environment-specific values are applied during launch. Heavy user-data scripts increase boot time, external dependencies, and partial-failure risk. User data is not a secret store.

11.3 IMDSv2 and instance identity

The Instance Metadata Service exposes instance identity and temporary credentials for the attached role. IMDSv2 uses a token-based session flow and should normally be required in the launch template. Disable the endpoint when it is unnecessary, use a low hop limit unless a documented container path requires more, and never persist temporary metadata credentials.

Applications should use current AWS SDK credential providers rather than calling metadata directly. Requiring IMDSv2 can expose old applications or agents that still use IMDSv1, so enforcement should include inventory, testing, and remediation.

11.4 Nitro, placement, capacity, and lifecycle

Modern EC2 families are commonly based on the Nitro System, which offloads networking, EBS, management, and isolation functions. Engineers should confirm that the chosen family supports required capabilities such as enhanced networking, high EBS bandwidth, hibernation, enclaves, bare metal, or Multi-Attach scenarios.

Cluster placement optimizes tightly coupled communication in one Availability Zone; partition placement separates distributed-system groups across partitions; spread placement separates a limited number of critical instances. None replaces multi-AZ resilience. Savings Plans and Reserved Instances primarily address cost, Capacity Reservations address matching zonal capacity, and Spot provides interruptible spare capacity.

Reboot restarts the guest. Stop/start releases compute and may move the instance. Hibernation writes memory to an encrypted root volume and stops, subject to support constraints. Termination removes the instance and volumes marked for deletion. Immutable fleets normally replace rather than preserve failed members.

EC2 fleet decision matrix

Decision	Use	Main risk
Golden AMI	Fast, consistent launches with pretested software	Image pipeline and patch cadence must be maintained
Launch template version	Controlled configuration promotion and rollback	Using latest without approval
Cluster placement	Low-latency tightly coupled workload	Single-AZ correlated failure
Capacity Reservation	Guaranteed matching capacity in an AZ	Charged while active and attributes must match
Spot capacity	Interruptible workers and diversified fleets	Capacity interruption and unavailable replacement

Enterprise Architecture

Versioned EC2 delivery

Source control: image, template, bootstrap, tests

V

Pipeline: build, patch, scan, integration test

V

Approved AMI and launch-template version

V

Auto Scaling refresh or blue/green replacement

CloudWatch, Systems Manager, CloudTrail, inventory, and runbooks

IMDSv2 token example

```
TOKEN=$(curl -sS -X PUT -H "X-aws-ec2-metadata-token-ttl-seconds: 21600"
http://169.254.169.254/latest/api/token)

curl -sS -H "X-aws-ec2-metadata-token: ${TOKEN}" http://169.254.169.254/latest/meta-data/instance-id
```

Production Best Practices

- Pin an approved launch-template version for production.
- Tag instances, volumes, and interfaces at creation.
- Use instance roles and short-lived credentials.
- Record bootstrap output and gate readiness on application health.
- Keep prior image/template artifacts through the rollback window.
- Separate price optimization from capacity assurance.

Troubleshooting

Symptom	Likely cause	Evidence / action
Instance remains pending	Capacity, placement, quota, or incompatible template	Read EC2 state reason; test another type/AZ; verify reservation match and quotas
Running but not ready	Bootstrap or application failure	Inspect cloud-init, bootstrap, systemd, target health, and security path
Application AccessDenied	Role, trust, policy, or KMS grant	Verify caller identity, attached profile, CloudTrail, and key policy
IMDS request fails	v2 required, endpoint disabled, or hop limit	Update application/SDK and validate metadata options
Poor network/storage performance	Instance bandwidth or workload limit	Correlate instance, ENA, EBS, and application metrics

Hands-on Lab: Secure versioned launch template

Objective: Create a launch template that requires IMDSv2, uses an instance profile, encrypts storage, tags resources, and completes bootstrap without public administration.

Implementation Tasks

1. Create a least-privilege instance role and approved Linux AMI.
2. Create template version 1 with encrypted gp3 root storage, required IMDSv2, monitoring, user data, and tag specifications.
3. Launch a private instance and connect through Session Manager.

4. Validate bootstrap logs, metadata token behavior, role permissions, metrics, and tags.
5. Create version 2 with a harmless application change and compare versions.
6. Promote the validated version only after evidence is captured.

Validation

- No inbound SSH/RDP rule exists.
- Tokenless metadata requests fail and token-based requests succeed.
- The role can perform the required action but not an unrelated write action.
- Instance and EBS volume have ownership and expiry tags.

Cleanup

- Terminate test instances.
- Delete unused template versions and temporary IAM resources.

Interview Questions

1. Why are launch templates safer than manual launches?
2. How do default and latest template versions differ?
3. When should software be baked into an AMI?
4. What risk does IMDSv2 reduce?
5. Compare cluster, partition, and spread placement.
6. Why does a Savings Plan not guarantee launch capacity?
7. How would you diagnose a running instance that never becomes healthy?

Chapter Summary

- Production EC2 is operated as a replaceable, versioned fleet.
- Approved AMIs, launch templates, roles, and IMDSv2 create a secure baseline.
- Placement, capacity, and pricing mechanisms solve different problems.
- Application readiness requires evidence beyond the EC2 running state.

Official AWS References

EC2 launch templates - <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-launch-templates.html>

Placement groups - <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/placement-groups.html>

Advanced launch-template settings - <https://docs.aws.amazon.com/autoscaling/ec2/userguide/advanced-settings-for-your-launch-template.html>

Advanced Amazon VPC

Address planning, subnet tiers, routing, egress, endpoints, IPv6, and hybrid-ready networks

VOLUME 6B | CHAPTER 12

Learning Objectives

- Create non-overlapping IPv4/IPv6 address plans.
- Design public, private, and isolated subnets across Availability Zones.
- Choose internet, NAT, endpoint, peering, and Transit Gateway paths.
- Troubleshoot complete forward and return routes.

12.1 VPC as a routing and ownership boundary

A VPC is a regional virtual network containing address ranges, subnets, route tables, gateways, endpoints, security controls, and connectivity. Subnets are Availability Zone resources. Good design begins with failure domains and required communications, not with the default wizard.

CIDR allocation must account for growth, containers, managed-service interfaces, future environments, hybrid networks, acquisitions, and regional expansion. Overlap makes peering, Transit Gateway, VPN, and Direct Connect routing difficult or impossible. Reserve space and use a governed IP address management process.

12.2 Subnet tiers and route evaluation

A public subnet has a route to an internet gateway; an IPv4 workload also needs a public address and security permission. A private subnet can initiate internet connections through NAT without accepting unsolicited inbound IPv4 sessions. An isolated subnet has no default internet route and uses only explicit private routes and endpoints.

AWS route selection uses longest-prefix match. A specific endpoint, peering, Transit Gateway, or appliance route overrides a broader default route. Troubleshooting should write the source, destination, protocol, forward route, policy checks, and return route before changing anything.

12.3 NAT, endpoints, and IPv6

A public NAT gateway resides in a public subnet and uses an Elastic IP. For zonal resilience, route each private subnet to a NAT gateway in the same Availability Zone. One centralized NAT can reduce resource count but creates a zonal dependency and cross-AZ data paths. NAT gateways have hourly and processing costs and do not accept security groups.

Gateway endpoints provide private route-table access to supported services such as S3 and DynamoDB. Interface endpoints create private network interfaces protected by security groups and are powered by PrivateLink. Endpoint policies restrict the endpoint path but do not replace IAM or resource policies.

IPv6 does not use NAT for address conservation. Dual-stack security must include IPv6 routes, security rules, logs, DNS, and tooling. Use an egress-only internet gateway for outbound-only IPv6 internet communication.

12.4 Multi-VPC, hybrid, and observability

VPC peering is simple point-to-point and non-transitive. Transit Gateway provides hub-and-spoke routing at scale. PrivateLink exposes a service privately without broad network reachability. VPN provides encrypted connectivity over the internet; Direct Connect supports dedicated private connectivity options. Every design must align routes on both sides and include DNS.

Enable VPC Flow Logs at a useful scope, centralize retention, and monitor rejected flows, NAT behavior, endpoint usage, IP exhaustion, and route changes. Reachability Analyzer evaluates configuration paths but does not prove that an application is listening. VPC Block Public Access can provide authoritative account-level guardrails where compatible with the architecture.

VPC connectivity matrix

Path	Best use	Key caution
Internet gateway	Direct public IPv4/IPv6 connectivity	Resource still needs address and security permission
NAT gateway	Outbound IPv4 from private subnets	Cost, zonal design, no inbound sessions
Gateway endpoint	Private S3/DynamoDB access	Route-table and endpoint-policy scope
Interface endpoint	Private access to many services	Per-AZ interfaces, security groups, hourly cost
VPC peering	Small number of direct private networks	Non-transitive and CIDRs cannot overlap
Transit Gateway	Many VPCs and hybrid routes	Route-table governance and cost

Enterprise Architecture

Highly available three-tier VPC

Route 53 / CloudFront / internet clients

V

ALB in public subnets across AZ A and AZ B

V

Application fleet in private subnets with same-AZ NAT and endpoints

V

Data services in isolated subnets with explicit tier policy

V

Flow logs, CloudTrail, Config, GuardDuty, and central operations

Production Best Practices

- Reserve address space and prevent overlap.
- Use one route table per routing intent, not per resource by habit.
- Prefer endpoints for required AWS service traffic where appropriate.
- Deploy zonal egress when the availability objective requires it.
- Design IPv6 policy and logging with parity.
- Centralize network evidence and detect unauthorized route changes.

Troubleshooting

Symptom	Likely cause	Evidence / action
Private instance has no internet	NAT/IGW route, EIP, or DNS missing	Trace private route to same-AZ NAT, NAT public route to IGW, and DNS
S3 traffic still uses NAT	Gateway endpoint not associated or DNS/route issue	Inspect endpoint route and private route-table association
Cross-VPC timeout	Missing forward/return route or overlap	Check both VPC/TGW route tables, CIDRs, SGs, and NACLs
IPv4 works but IPv6 fails	Missing AAAA, ::/0, or IPv6 rule	Inspect DNS, egress-only gateway, SG/NACL, and application binding
Subnet out of addresses	Under-sized CIDR or leaked interfaces	Inventory ENIs, endpoints, load balancers, and redesign capacity

Hands-on Lab: Two-AZ VPC with controlled egress

Objective: Build mirrored public, private, and isolated tiers; use zonal NAT gateways and an S3 gateway endpoint; prove the intended paths with logs.

Implementation Tasks

1. Choose a non-overlapping VPC CIDR and reserve future ranges.
2. Create mirrored subnet tiers in two Availability Zones.
3. Attach an internet gateway and create explicit route tables.
4. Create one NAT gateway per zone and same-AZ private routes.
5. Create and associate an S3 gateway endpoint.
6. Launch a private Session Manager test instance and enable flow logs.
7. Remove the NAT default route temporarily and prove S3 endpoint access remains while general internet fails.

Validation

- The instance has no public IP or management port.
- S3 access follows the endpoint route.

- General IPv4 egress follows same-AZ NAT.
- Isolated subnets have no default internet route.
- Flow logs show accepted and rejected tests.

Cleanup

- Terminate test instances and remove endpoints, NAT gateways, Elastic IPs, logs, subnets, routes, and VPC.
- Verify no orphan Elastic IP or log group remains.

Interview Questions

1. What makes a subnet public?
2. Why is a NAT gateway placed in a public subnet?
3. How does longest-prefix match work?
4. When is one NAT gateway per AZ justified?
5. Compare gateway and interface endpoints.
6. Why can dual stack create a security blind spot?
7. When would you choose PrivateLink over peering?
8. How would you troubleshoot a timeout across VPCs?

Chapter Summary

- A VPC is an address, routing, security, ownership, and observability system.
- Routes and addresses determine actual connectivity, not subnet names.
- Endpoints reduce public-path dependency; NAT requires resilience and cost analysis.
- Reliable troubleshooting follows both forward and return paths.

Official AWS References

VPC subnet basics - <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-subnet-basics.html>

NAT gateways - <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-gateway.html>

Egress-only internet gateway - <https://docs.aws.amazon.com/vpc/latest/userguide/egress-only-internet-gateway.html>

Security Groups, Network ACLs, and Segmentation

Least-privilege network policy, managed administration, egress control, and evidence-driven troubleshooting

VOLUME 6B | CHAPTER 13

Learning Objectives

- Explain stateful security groups and stateless NACLs.
- Use security-group references for tier-to-tier policy.
- Apply NACLs as coarse defense in depth.
- Replace public management ports with controlled access.
- Troubleshoot denied traffic without broad rules.

13.1 Multiple enforcement layers

A packet may be affected by routing, security groups, network ACLs, load-balancer listeners, endpoint policies, appliance rules, host firewalls, and application authorization. Assign each layer a purpose instead of duplicating every rule. Least privilege defines the required source, destination, protocol, and port; a rule created only to make a test pass is technical debt.

13.2 Stateful security groups

Security groups attach to network interfaces, contain allow rules, and are stateful: return traffic for an allowed connection is recognized without a reverse rule. They have no explicit deny. The preferred pattern is client to ALB on HTTPS, ALB security group to application port, and application security group to database port.

A referenced security group identifies associated private interfaces for supported flows; it does not create routing and is not group nesting. References are more stable than hard-coded addresses and clearly express workload relationships.

13.3 Stateless network ACLs

NACLs associate with subnets and evaluate ordered inbound and outbound allow/deny rules. Because they are stateless, request and return traffic, including client ephemeral ports, must be allowed. Complex NACLs often create outages. Use them for coarse subnet guardrails or known denials while security groups carry workload-specific policy.

13.4 Administrative access and egress

Public SSH/RDP should normally be replaced by Systems Manager Session Manager, a private operations network, Client VPN, or another controlled path. Session Manager still requires least-privilege IAM, agent/connectivity health, encrypted log delivery, and break-glass procedures.

Mature platforms reduce unrestricted outbound traffic with endpoints, security-group destinations, managed prefix lists, proxies, DNS controls, and centralized inspection. Discover real dependencies through logs before enforcing restrictions, and provide approved paths for time, certificate validation, package repositories, monitoring, and incident tools.

13.5 Segmentation and Zero Trust

Microsegmentation reduces reachable paths but does not replace identity. A practical Zero Trust posture combines small network blast radius with workload identities, service authentication, device/user context, secrets management, continuous logging, and explicit authorization.

Security control comparison

Security group	Network ACL	Primary use
Stateful	Stateless	Connection-aware workload policy vs subnet guardrail
Allow rules only	Allow and deny rules	Fine-grained allow vs coarse deny/allow
All rules considered	Numeric first-match order	Simple policy vs carefully ordered rules
Attached to ENIs	Associated with subnets	Application tiers vs subnet boundary
Return traffic recognized	Return/ephemeral ports required	Lower operational risk vs more troubleshooting complexity

Enterprise Architecture

Tier policy flow

Client -> ALB security group on TCP 443

▼

ALB -> application security group on application port

▼

Application -> database security group on database port

▼

Administration through Session Manager; no public SSH/RDP

▼

Flow logs, session logs, CloudTrail, and alerts

Production Best Practices

- Use security-group references for application tiers.
- Keep default-deny and remove obsolete rules.
- Use NACLs sparingly with documented ephemeral-port behavior.
- Eliminate 0.0.0.0/0 management access.
- Restrict egress progressively after dependency discovery.
- Test denied paths and preserve flow-log evidence.

Troubleshooting

Symptom	Likely cause	Evidence / action
Timeout	Route or SG source/destination mismatch	Confirm actual ENIs, addresses, forward/return routes, and SG references
One direction works	NACL return or asymmetric route	Review ordered inbound/outbound rules and ephemeral ports
ALB target unhealthy	Target SG or health path blocked	Check source ALB SG, target port, binding, and health response
Session Manager unavailable	Agent, role, DNS, endpoint, or outbound path	Validate SSM agent, instance profile, interface-endpoint SG, and logs
Rule appears correct	Traffic uses IPv6 or different path	Inspect resolved address, interface, and flow logs

Hands-on Lab: Tier-to-tier security-group references

Objective: Allow only ALB-to-application and application-to-database traffic while using Session Manager for administration.

Implementation Tasks

1. Create ALB, application, and database security groups.
2. Allow HTTPS to the ALB from the approved test source.
3. Allow the app port only from the ALB security group.
4. Allow the database test port only from the app security group.
5. Remove inbound management ports and connect through Session Manager.
6. Apply a deliberately restrictive test NACL, observe failure, then correct request and return rules.
7. Document the final traffic matrix.

Validation

- Direct client access to application and database ports fails.

- Expected tier flows succeed.
- Session Manager works without inbound SSH/RDP.
- Flow logs distinguish allowed and denied tests.

Cleanup

- Delete temporary listeners, instances, load balancer resources, groups, and test NACLs.
- Restore original subnet associations.

Interview Questions

1. Why are security groups stateful?
2. How does NACL rule ordering work?
3. Why are ephemeral ports important?
4. Why use a security-group reference instead of a CIDR?
5. Why is Session Manager safer than a public bastion?
6. How would you introduce egress restrictions safely?
7. Does a security-group reference create routing?
8. How do you prove whether routing, SG, or NACL caused a timeout?

Chapter Summary

- Security groups are the primary stateful workload firewall.
- NACLs are stateless subnet defense in depth and should remain simple.
- Managed private administration reduces exposed attack surface.
- Evidence-driven troubleshooting is safer than broad temporary access.

Official AWS References

VPC security groups - <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-security-groups.html>

Network ACLs - <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-network-acls.html>

VPC network inventory - <https://docs.aws.amazon.com/vpc/latest/userguide/vpc-network-inventory.html>

Amazon Route 53 and DNS Engineering

Authoritative DNS, routing policies, health evaluation, private resolution, and safe change operations

VOLUME 6B | CHAPTER 14

Learning Objectives

- Explain recursive and authoritative DNS resolution.
- Design public and private hosted zones with clear ownership.
- Choose Route 53 routing policies for the actual resilience objective.
- Use aliases, health checks, Resolver endpoints, and query evidence correctly.
- Operate DNS changes with tested rollback.

14.1 DNS is a distributed control plane

Applications depend on DNS before they can reach a load balancer, API, database, or service endpoint. A resolver follows delegation from the root to a top-level domain and then to authoritative name servers. Caches retain answers according to the record time to live, so a change is not globally instantaneous.

Route 53 is both an authoritative DNS service and a routing control plane. Reliability requires control of registration, delegation, hosted-zone records, certificates, health signals, query logging, and change ownership. A correct record in the wrong hosted zone does nothing; a correct hosted zone without registrar delegation is not authoritative.

14.2 Hosted zones, records, and aliases

A public hosted zone answers internet DNS queries. A private hosted zone answers only through associated VPCs and supported hybrid resolution paths. Split-horizon DNS can intentionally return different answers internally and externally, but identical names across several private zones can create confusing precedence.

Common records include A and AAAA for addresses, CNAME for canonical names, MX for mail, TXT for verification and policy, and NS/SOA records for authority. Route 53 alias records can point the zone apex or another name to selected AWS resources without the CNAME apex restriction. An alias target and a traditional CNAME are not interchangeable.

14.3 Routing policies and health

Simple routing returns one record set. Weighted routing distributes answers by configured weights and is useful for gradual release, experiments, or controlled migration. Latency routing chooses the configured Region expected to give lower latency. Geolocation uses user location, geoproximity considers resource location and bias, multivalue answers can return several healthy records, and failover routing selects primary or secondary based on health.

DNS routing does not proxy traffic and does not guarantee that an exact percentage of requests follows each answer because resolvers cache and aggregate queries. Health checks should test the customer-relevant path where possible. Evaluate Target Health can inherit health from supported alias targets, while explicit Route 53 health checks can monitor endpoints or calculated combinations.

14.4 Hybrid DNS with Route 53 Resolver

Inbound Resolver endpoints allow on-premises resolvers to query selected private AWS namespaces. Outbound endpoints and forwarding rules send specified domains from VPCs to external resolvers. Rules can be shared across accounts through AWS Resource Access Manager. Endpoint security groups, availability across subnets, resolver capacity, logging, and loop prevention are production concerns.

Avoid forwarding a domain in a circle between AWS and on-premises DNS. Define one authoritative owner for each namespace, document delegation and forwarding, and test resolution from every relevant network segment.

14.5 Safe DNS change management

Lowering TTL shortly before a planned migration can reduce cache persistence, but only after the previous higher TTL has expired. Changes should include prechecks, staged records, certificate validation, health readiness, explicit observation windows, rollback records, and post-change verification from multiple resolvers. DNSSEC can protect authenticity where required but adds key and delegation lifecycle responsibilities.

Route 53 routing policy selection

Policy	Best fit	Caution
Simple	Single endpoint or multiple unordered answers	No health-aware traffic policy by itself
Weighted	Canary, migration, active-active distribution	Observed request share differs because of caching
Latency	Multi-Region user latency optimization	Requires healthy regional capacity and data strategy
Failover	Active-passive primary and recovery endpoint	Health check and recovery environment must be tested
Geolocation / geoproximity	Location-sensitive content or compliance routing	Default coverage and data correctness still required
Multivalue	Several healthy endpoint answers	Not a replacement for a full load balancer

Enterprise Architecture

Hybrid authoritative DNS

Public registrar delegates example.com to Route 53 public hosted zone

V

Public aliases direct users to CloudFront or regional load balancers

V

Private hosted zones provide internal service names to associated VPCs

V

Inbound and outbound Resolver endpoints bridge approved on-premises namespaces

V

Query logs, health alarms, CloudTrail, and change runbooks provide evidence

DNS verification commands

```
dig +trace example.com

dig A app.example.com
dig AAAA app.example.com
dig NS example.com

aws route53 list-hosted-zones
aws route53 list-resource-record-sets --hosted-zone-id ZONE_ID
```

Production Best Practices

- Separate registration, hosted-zone administration, and application record permissions.
- Use aliases for supported AWS targets and zone-apex names.
- Document authoritative ownership for every public and private suffix.
- Design health checks around user-visible dependencies and avoid false failover.
- Use low TTLs selectively; extremely low TTL increases query load and does not eliminate caching.
- Log resolver queries where investigation or compliance requires them.
- Test delegation, certificates, IPv4/IPv6 answers, and rollback before a migration.

Troubleshooting

Symptom	Likely cause	Evidence / action
NXDOMAIN	Record absent, wrong zone, or private zone not visible	Check queried name, hosted-zone ID, VPC association, and authoritative NS
Old answer persists	Resolver or application cache retained previous TTL	Inspect TTL and query several recursive resolvers; wait for expiry
Failover does not occur	Health check not associated, inverted, or testing wrong path	Inspect check status, alarm history, record policy, and endpoint dependencies
Internal users resolve public address	Private zone/Resolver association or forwarding missing	Verify VPC association, rules, endpoint security groups, and query path

Symptom	Likely cause	Evidence / action
Intermittent resolution	Resolver loop, endpoint capacity, or inconsistent delegation	Review Resolver logs, forwarders, NS glue, and network health
IPv6 clients fail	Missing AAAA or target/network IPv6 readiness	Verify record, listener, subnet route, SG/NACL, and application binding

Hands-on Lab: Health-aware DNS migration

Objective: Build a test namespace with weighted and failover records, verify TTL behavior, and practice a reversible migration.

Implementation Tasks

1. Create a disposable public or private hosted zone.
2. Create two test endpoints or static responses and verify direct health.
3. Add weighted records and observe answers with repeated dig queries.
4. Attach health evaluation or Route 53 health checks as appropriate.
5. Prepare a failover record set with primary and secondary endpoints.
6. Simulate primary failure, record detection and answer change, then restore.
7. Document TTL, ownership, validation evidence, and rollback steps.

Validation

- Authoritative NS and SOA are correct.
- Weighted answers are visible without claiming exact request percentages.
- Unhealthy targets are excluded according to the selected policy.
- Primary recovery is observed and no stale record remains after the test.

Cleanup

- Delete test records, health checks, endpoints, and hosted zone.
- Confirm registrar delegation was not left pointing to a deleted public zone.

Interview Questions

1. What is the difference between recursive and authoritative DNS?
2. Why can DNS changes appear delayed?
3. Alias versus CNAME: when do you use each?
4. How does weighted routing differ from load balancing?
5. When would latency routing be unsafe?
6. How do private hosted zones resolve from on-premises networks?
7. What causes a DNS forwarding loop?
8. How would you perform a low-risk DNS cutover?

Chapter Summary

- DNS is a cached distributed control plane, not an instantaneous database.
- Hosted-zone authority, delegation, records, health, and ownership must align.
- Routing policies solve different business objectives and inherit caching limits.
- Hybrid DNS requires deliberate namespace ownership and loop-free forwarding.

Official AWS References

Route 53 routing policies - <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/routing-policy.html>

Alias records - <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/routing-to-elb-load-balancer.html>

Route 53 Resolver - <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/resolver.html>

Elastic Load Balancing Deep Dive

Application, transport, and appliance traffic distribution with health, TLS, observability, and deployment safety

VOLUME 6B | CHAPTER 15

Learning Objectives

- Select ALB, NLB, or GWLB from protocol and architecture requirements.
- Configure listeners, rules, target groups, health checks, and TLS safely.
- Explain connection draining, client address behavior, and cross-zone implications.
- Use load-balancer evidence to troubleshoot availability and latency.
- Design blue/green and canary release patterns.

15.1 Why a load balancer is more than a traffic splitter

Elastic Load Balancing provides stable service endpoints while backends are added, removed, replaced, or scaled. Its production responsibilities include protocol termination, policy enforcement, target health, connection lifecycle, access evidence, fault-domain distribution, and integration with DNS, certificates, web application protection, and Auto Scaling.

A load balancer cannot make an unhealthy application healthy. Its health model, backend readiness, dependency limits, and deployment behavior must reflect what users actually need.

15.2 Application Load Balancer

ALB operates at the HTTP/HTTPS application layer. Listeners evaluate ordered rules that can match host, path, method, header, source IP, and query conditions, then forward, redirect, return a fixed response, or authenticate through supported integrations. Multiple target groups enable service routing and weighted forwarding.

TLS certificates are managed through listener configuration, commonly using AWS Certificate Manager. Security policies control supported protocol and cipher behavior. The target must listen on the registered port and return an expected health response; a process that merely accepts TCP may still be functionally unhealthy.

15.3 Network and Gateway Load Balancers

NLB operates at the transport layer for high-performance TCP, UDP, and TLS workloads and can provide static zonal addresses. It preserves source characteristics differently from ALB and is appropriate when application-layer routing is not required.

GWLB distributes traffic to compatible virtual network appliances through a transparent bump-in-the-wire pattern using GENEVE encapsulation. It supports scalable inspection fleets but requires symmetric routing, appliance readiness, route-table design, and careful failure behavior. It is not a normal web load balancer.

15.4 Target health and connection lifecycle

Target groups define target type, protocol, port, health checks, deregistration behavior, and optional attributes. Health checks should be inexpensive, dependency-aware, and resistant to false positives. A shallow check may route users to a broken service; an overly deep check can cause a shared dependency failure to remove every target.

Deregistration delay allows in-flight work to complete before a target is removed. Application shutdown must stop accepting new work, complete or terminate existing work within the allowed period, and publish useful logs. Idle timeouts, keep-alive behavior, HTTP/2 or gRPC characteristics, and long-lived connections affect perceived availability during deployment.

15.5 Observability and releases

Key evidence includes target-health reason codes, request count, response codes, target response time, rejected connections, processed bytes, access logs, application logs, and DNS answers. Distinguish load-balancer-generated errors from target-generated errors.

Blue/green deployment uses separate old and new target populations. Traffic can shift through listener rules, weighted target groups, or DNS depending on the design. A canary begins with limited traffic, compares service-level indicators and business metrics, and automatically or manually rolls back when guardrails fail.

Load balancer selection

Type	Use when	Avoid assuming
ALB	HTTP/HTTPS, host/path routing, web and API services	Static IP or arbitrary TCP/UDP semantics
NLB	TCP/UDP/TLS, high throughput, source/network requirements	Layer-7 routing or content inspection
GWLb	Scale-out third-party virtual appliances	Direct application delivery
No ELB	Single-purpose internal test or service-native alternative	That a single endpoint meets production availability

Enterprise Architecture

Multi-AZ application delivery

Route 53 alias -> internet-facing ALB across two or more subnets

V

HTTPS listener -> ordered host/path rules and certificate policy

V

Target groups -> private Auto Scaling instances in multiple Availability Zones

V

Application -> downstream services through least-privilege security groups

V

Access logs, CloudWatch metrics, WAF signals, target health, and deployment alarms

Target-health inspection

```
aws elbv2 describe-load-balancers
aws elbv2 describe-listeners --load-balancer-arn LOAD_BALANCER_ARN
aws elbv2 describe-rules --listener-arn LISTENER_ARN
aws elbv2 describe-target-health --target-group-arn TARGET_GROUP_ARN
```

Production Best Practices

- Choose the load balancer from protocol and policy needs, not familiarity.
- Deploy load-balancer nodes and healthy targets across multiple Availability Zones.
- Use HTTPS, approved policies, managed certificates, and automated renewal monitoring.
- Design health checks with explicit success criteria and separate readiness from liveness where possible.
- Align deregistration delay with application shutdown and request duration.
- Enable access logs and preserve correlation identifiers across tiers.
- Protect public HTTP services with appropriate WAF, rate, and authentication controls.
- Exercise target loss, zonal loss, certificate replacement, and rollback.

Troubleshooting

Symptom	Likely cause	Evidence / action
503 from ALB	No healthy or registered targets	Inspect target health, registration, health path, port, and deployment status
502 from ALB	Target closed connection or invalid response	Inspect application/proxy logs, protocol, headers, and timeout behavior
504 from ALB	Target or dependency exceeded timeout	Measure target response time and downstream latency; review timeout budgets
Healthy check but users fail	Health path too shallow or listener rule mismatch	Test user path and rule priority; enrich readiness check
Targets drain too slowly	Long-lived connections or deregistration delay	Inspect connection duration and graceful shutdown design
Uneven traffic	Sticky sessions, long connections, zonal target distribution, or caching	Review attributes, client behavior, target count, and metrics by zone
NLB flow fails	Target health, route, NACL, source behavior, or protocol mismatch	Trace transport flow and confirm target-type requirements

Hands-on Lab: ALB blue/green deployment

Objective: Deploy two application target groups, shift traffic safely, observe health and logs, and roll back without editing instances.

Implementation Tasks

1. Create a private launch template or two small test target groups with distinct responses.
2. Create an ALB across two public subnets and an HTTPS or lab HTTP listener.
3. Configure ALB-to-target security-group references.
4. Verify target health and access logs.
5. Create a listener rule or weighted forwarding action for blue and green groups.
6. Shift a small percentage to green, inspect errors and latency, then complete or roll back.
7. Deregister blue and observe connection draining.

Validation

- Only the load balancer is internet reachable.
- Both target groups show the expected health state and reason codes.
- Traffic shift is visible in responses and metrics.
- Rollback restores the blue response within the designed control window.
- Deregistered targets stop receiving new requests.

Cleanup

- Delete listener rules, target groups, load balancer, test instances/ASGs, log objects, and security groups.
- Verify public IPv4 and certificate test resources are removed or retained intentionally.

Interview Questions

1. Compare ALB, NLB, and GWLB.
2. What does a target group contain?
3. Why can a healthy target still fail user requests?
4. What is deregistration delay?
5. How do you distinguish a load-balancer 5xx from a target 5xx?
6. When is stickiness harmful?
7. How would you implement a canary release?
8. What routing concern is critical for GWLB appliances?

Chapter Summary

- ALB, NLB, and GWLB solve different protocol and architecture problems.
- Health checks, connection lifecycle, TLS, and logs determine operational quality.
- Load balancers expose failure; they do not repair poor application readiness.
- Safe releases require isolated target populations, guardrails, and rollback.

Official AWS References

Elastic Load Balancing product comparison - <https://docs.aws.amazon.com/elasticloadbalancing/latest/userguide/what-is-load-balancing.html>

Application Load Balancers - <https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html>

Gateway Load Balancers - <https://docs.aws.amazon.com/elasticloadbalancing/latest/gateway/introduction.html>

EC2 Auto Scaling Engineering

Demand-based capacity, mixed instance fleets, lifecycle automation, instance refresh, and stability controls

VOLUME 6B | CHAPTER 16

Learning Objectives

- Design Auto Scaling groups around replaceable capacity and measurable demand.
- Compare target tracking, step, scheduled, and predictive scaling.
- Configure warmup, cooldown, health checks, termination policy, and lifecycle hooks.
- Use mixed instance policies and Spot diversification safely.
- Perform controlled instance refresh and diagnose scaling instability.

16.1 Auto Scaling is a feedback-control system

An Auto Scaling group maintains desired capacity between minimum and maximum limits and replaces unhealthy instances. Scaling adds or removes capacity in response to a signal. The system is stable only when the metric represents demand, new capacity becomes useful within the assumed time, dependencies can absorb growth, and scale-in does not remove needed work.

High availability begins with subnets in multiple Availability Zones, a launch template, health evaluation, and deterministic bootstrapping. Auto Scaling is not a substitute for application statelessness, database capacity planning, queue design, or regional disaster recovery.

16.2 Scaling policy selection

Target tracking adjusts capacity to keep a metric near a target and is the normal starting point for proportional workloads. Step scaling changes capacity based on alarm breach severity. Scheduled scaling handles known business events, while predictive scaling forecasts recurring demand and can provision ahead of it. Multiple policies can coexist, but their interactions must be understood.

Good metrics scale in proportion to load per unit of capacity: average CPU for CPU-bound workers, requests per target, queue backlog per worker, or a custom concurrency metric. Raw total request count often rises with capacity and can create the wrong control behavior.

16.3 Warmup, health, and lifecycle

Instance warmup prevents new capacity from immediately distorting scaling metrics before it is ready. Health checks can include EC2 state and load-balancer target health. Grace periods should cover normal startup without hiding sustained failure.

Lifecycle hooks pause launches or terminations for controlled actions such as configuration validation, registration, log export, queue draining, or security cleanup. Hooks require timeouts, heartbeats where needed, idempotent handlers, and a defined abandon/continue decision. Warm pools can reduce readiness time for slow-start applications but add state, patching, and cost considerations.

16.4 Mixed instances, Spot, and scale-in

A mixed instances policy can combine instance types and On-Demand/Spot capacity. Select types with comparable application capacity, use attribute-based or explicit requirements, diversify Spot pools, and make interruption handling part of the application. Capacity-optimized allocation often reduces interruption risk compared with selecting only the lowest current price.

Scale-in uses termination policies and may rebalance Availability Zones before other preferences. Protect critical temporary work only with a clear release mechanism. A group that cannot terminate protected members can remain above desired capacity. Stateful work should checkpoint externally rather than depend on indefinite instance protection.

16.5 Instance refresh and safe fleet change

Instance refresh replaces members to a new launch-template or image version while respecting minimum healthy percentage and optional checkpoints. Bake and test the artifact first, use alarms or deployment gates, and preserve an old approved version for rollback.

Scaling activities provide the authoritative operational history: launches, failures, health replacements, rebalance actions, and termination decisions. Investigation should begin with activities, policy state, metrics, alarms, target health, launch-template version, quotas, and capacity availability.

Scaling mechanism comparison

Mechanism	Best fit	Main design concern
Target tracking	Continuous proportional demand	Metric must represent load per capacity unit
Step scaling	Severity-based response	Alarm thresholds and overlapping policies
Scheduled scaling	Known event or business timetable	Calendar drift and unexpected demand
Predictive scaling	Recurring forecastable patterns	Forecast quality and minimum-capacity planning
Manual desired change	Incident or controlled maintenance	Must not become normal untracked operation

Enterprise Architecture

Elastic application fleet

Demand metric and service-level indicators -> CloudWatch

V

Scaling policy -> desired capacity within min/max guardrails

V

Auto Scaling group -> approved launch-template version across multiple AZs

Lifecycle/health -> load balancer registration, readiness, draining, replacement

Scaling activities, alarms, logs, costs, and deployment pipeline -> operators

Auto Scaling inspection

```
aws autoscaling describe-auto-scaling-groups --auto-scaling-group-names APP_ASG
aws autoscaling describe-policies --auto-scaling-group-name APP_ASG
aws autoscaling describe-scaling-activities --auto-scaling-group-name APP_ASG
aws autoscaling describe-instance-refreshes --auto-scaling-group-name APP_ASG
```

Production Best Practices

- Set minimum capacity from availability requirements, not average load alone.
- Use a demand metric that decreases when useful capacity is added.
- Measure real startup and readiness time before setting warmup.
- Diversify compatible Spot types and handle interruption signals.
- Set explicit dependency ceilings so scaling does not overload databases or third parties.
- Use lifecycle hooks only with timeout, retry, idempotency, and dead-letter behavior.
- Perform instance refresh with health guardrails and rollback evidence.
- Alarm on repeated launch failures, no healthy targets, capacity limits, and unstable oscillation.

Troubleshooting

Symptom	Likely cause	Evidence / action
ASG will not scale out	Policy inactive, alarm state wrong, max reached, or metric missing	Inspect policy, alarm, desired/min/max, activities, and metric dimensions
Launches repeatedly fail	AMI/template, IAM, subnet IP, quota, or capacity issue	Read scaling activity cause and EC2 launch failure details
Fleet oscillates	Poor metric, short evaluation, or warmup/cooldown mismatch	Graph demand and capacity; lengthen stabilization and redesign signal
Instances healthy in EC2 but replaced	ELB health check or readiness failure	Inspect target reason codes, health type, grace period, and app logs
Scale-in stalls	All instances protected or lifecycle hook waiting	Inspect protection, hooks, heartbeat, and timeout action
Instance refresh pauses	Alarm, checkpoint, capacity, or unhealthy percentage	Inspect refresh status/reason, activities, and target health
Database overloads after scaling	Compute scales faster than downstream capacity	Add connection control, queue/backpressure, cache, and dependency guardrails

Hands-on Lab: Stable request-based Auto Scaling

Objective: Create a multi-AZ fleet behind an ALB, scale on demand, test replacement and instance refresh, and document control-loop behavior.

Implementation Tasks

1. Create a launch-template version with IMDSv2, role, bootstrap, and tags.
2. Create an ASG in at least two private subnets and attach an ALB target group.
3. Configure minimum, desired, and maximum capacity plus target tracking on requests per target or a suitable lab metric.
4. Generate controlled load and record scale-out time, readiness, and metric response.
5. Stop the application on one member and observe health replacement.
6. Publish a new template version and run an instance refresh with a health guardrail.
7. Reduce load and observe scale-in and deregistration.

Validation

- Capacity remains within min/max and spans configured Availability Zones.
- New targets become healthy before serving expected load.
- Unhealthy capacity is replaced automatically.
- Instance refresh replaces the old version without exceeding the error guardrail.
- Scaling activities explain every capacity change.

Cleanup

- Delete policies, ASG, launch-template test versions, ALB resources, instances, and logs.
- Confirm lifecycle hooks, warm pools, and public resources are absent.

Interview Questions

1. What makes a good target-tracking metric?
2. Warmup versus cooldown: what problem does each solve?
3. How does an ASG decide an instance is unhealthy?
4. Why diversify Spot capacity?
5. What is a lifecycle hook?
6. How can scale-in protection cause excess capacity?
7. How does instance refresh reduce deployment risk?
8. Why can aggressive compute scaling damage a database?

Chapter Summary

- Auto Scaling is a feedback system whose stability depends on metrics, readiness, and limits.
- Multi-AZ replaceable capacity is the foundation; policies modify desired capacity.
- Mixed fleets and lifecycle automation require interruption and timeout handling.

- Scaling activities and service metrics provide the evidence for diagnosis.

Official AWS References

Amazon EC2 Auto Scaling concepts - <https://docs.aws.amazon.com/autoscaling/ec2/userguide/what-is-amazon-ec2-auto-scaling.html>

Scaling policy types - https://docs.aws.amazon.com/autoscaling/ec2/userguide/scaling_plan.html

Instance refresh - <https://docs.aws.amazon.com/autoscaling/ec2/userguide/asg-instance-refresh.html>

Amazon EBS Deep Dive

Block-storage architecture, performance engineering, encryption, snapshots, resizing, and operational recovery

VOLUME 6B | CHAPTER 17

Learning Objectives

- Choose an EBS volume type from workload evidence.
- Explain IOPS, throughput, latency, queue depth, and instance limits.
- Design encryption and snapshot controls with AWS KMS.
- Resize and migrate volumes safely.
- Use Multi-Attach and recovery workflows only where technically appropriate.

17.1 EBS architecture and failure boundaries

Amazon Elastic Block Store provides durable block devices for EC2. A volume is created in one Availability Zone and attaches to compatible instances in that zone. The guest operating system creates partitions, file systems, volume managers, databases, and application-level consistency on top of the device. EBS durability does not eliminate corruption, accidental deletion, ransomware, or application-consistency requirements.

A resilient application should separate replaceable compute from persistent data, encrypt by default, monitor performance, create recoverable points, and practice restore. Multi-AZ service availability usually requires replication at the database or application layer, not an assumption that one EBS volume spans zones.

17.2 Volume families and selection

General Purpose SSD gp3 is the normal starting point for boot volumes and many transactional workloads because capacity, baseline performance, IOPS, and throughput can be adjusted independently within service constraints. Provisioned IOPS SSD io2 is intended for latency-sensitive, I/O-intensive databases and applications needing higher durability and predictable provisioned performance.

Throughput Optimized HDD st1 is suited to large sequential, frequently accessed data such as log processing and big-data streams. Cold HDD sc1 targets infrequently accessed sequential workloads at low cost. HDD volumes are not boot volumes and are poor choices for small random I/O. Selection must consider access size, read/write mix, latency, peak queue depth, burst behavior, and recovery requirements rather than storage size alone.

17.3 Performance is an end-to-end budget

Application latency includes filesystem, operating system, device, EBS service, network path, and instance EBS bandwidth. A volume can be provisioned above what the instance can deliver, or an instance can have unused bandwidth because the volume is undersized. CloudWatch metrics should be interpreted with application measurements and operating-system evidence such as iostat.

IOPS measures operations, throughput measures bytes per second, and latency measures time per operation. Approximate throughput equals IOPS multiplied by average I/O size until another limit is reached. Queue depth can improve device utilization but excessive sustained queues indicate saturation and increase response time. Credit-based behavior on applicable resources must be included in load tests.

17.4 Encryption and snapshots

EBS encryption protects data at rest, data moving between the instance and volume, snapshots, and volumes created from encrypted snapshots. The KMS key policy, grants, account boundary, and lifecycle become part of recoverability. Enabling encryption by default in each Region reduces unencrypted drift.

Snapshots are incremental service-managed recovery points stored in Amazon S3 infrastructure, but they are managed through EBS APIs rather than as normal S3 objects. A crash-consistent snapshot captures blocks at a moment; application-consistent recovery may require flushing, freezing, database backup integration, or coordinated multi-volume snapshots. Fast Snapshot Restore or pre-initialization may be needed when immediate full performance after restore matters.

17.5 Modification, replacement, and Multi-Attach

Elastic Volumes can modify type, size, IOPS, or throughput for many supported volumes without detaching. The operating system still needs partition, LVM, and filesystem expansion. Shrinking generally requires creating a smaller target and copying data. Every change needs a backup, capacity check, application test, and rollback plan.

EBS Multi-Attach is limited to supported Provisioned IOPS configurations and same-AZ Nitro-based instances. It exposes one block device to multiple hosts; it does not provide a distributed lock manager or cluster-safe filesystem. Use only with software designed for coordinated shared block access and test fencing to prevent corruption.

EBS volume decision matrix

Volume	Typical workload	Primary caution
gp3	Boot, application, moderate transactional data	Validate provisioned IOPS/throughput and instance ceiling
io2	Critical low-latency databases and high I/O	Higher cost; application must benefit from provisioned performance
st1	Large sequential streams and frequently read throughput data	Not for boot or random small I/O
sc1	Cold sequential data with infrequent access	Lowest performance; recovery and access time expectations
Instance store	Ephemeral scratch, cache, replicated data	Data is tied to instance lifecycle and must be disposable

Enterprise Architecture

Recoverable EBS workload

EC2 instance role and encrypted EBS volumes in one Availability Zone

V

Application-consistent snapshot orchestration and retention policy

V

Cross-account or cross-Region copy where recovery objectives require it

V

Restore test -> new volume -> filesystem/database validation -> application recovery

V

Metrics, KMS audit, AWS Backup/CloudTrail evidence, and runbooks**Volume performance inspection**

```
aws ec2 describe-volumes --volume-ids vol-EXAMPLE
aws ec2 describe-volumes-modifications --volume-ids vol-EXAMPLE
aws ec2 describe-snapshots --owner-ids self

# Linux guest evidence
lsblk -f
iostat -xz 1
```

Production Best Practices

- Start with measured workload requirements and include the EC2 EBS bandwidth ceiling.
- Encrypt by default using governed KMS keys and tested restore permissions.
- Separate boot, application, log, and database volumes when lifecycle or performance differs.
- Use application-consistent snapshot procedures for stateful systems.
- Tag snapshots with workload, source, retention class, owner, and recovery purpose.
- Monitor latency, operations, throughput, queue, burst/credit indicators, filesystem space, and application SLOs.
- Test restored performance, not merely snapshot completion.
- Use Multi-Attach only with cluster-aware software and fencing.

Troubleshooting

Symptom	Likely cause	Evidence / action
High storage latency	Volume or instance limit, queue saturation, filesystem, or application pattern	Correlate EBS metrics, iostat, I/O size, queue, instance bandwidth, and app traces
Volume attached but absent	Device naming, NVMe mapping, OS scan, or driver issue	Use lsblk/NVMe tools, dmesg, and instance attachment data

Symptom	Likely cause	Evidence / action
Filesystem did not grow	Only EBS size changed	Extend partition/LVM and filesystem with platform-specific commands
Snapshot restore initially slow	Blocks loading on first access	Pre-initialize or use Fast Snapshot Restore when justified
Cannot decrypt restored volume	KMS policy/grant/account/Region issue	Inspect key state, policy, caller, snapshot permissions, and CloudTrail
Multi-Attach corruption risk	Non-cluster-aware filesystem or missing fencing	Stop writes, preserve evidence, validate cluster design before reuse
Snapshot succeeds but app restore fails	Crash-consistent point lacks transaction coordination	Implement database-native or freeze/quiesce workflow and retest

Hands-on Lab: Encrypted EBS lifecycle and restore test

Objective: Provision a gp3 data volume, measure it, snapshot it consistently, modify performance, and restore to a separate validation instance.

Implementation Tasks

1. Create an encrypted gp3 volume in the test instance Availability Zone.
2. Attach, identify the device, create a filesystem, mount by UUID, and write test data.
3. Capture baseline application and iostat/CloudWatch evidence.
4. Quiesce writes and create a tagged snapshot.
5. Modify gp3 size or performance within a controlled limit and extend the guest filesystem.
6. Create a new encrypted volume from the snapshot and attach it to a separate validation instance.
7. Mount read-only first, verify data and application consistency, then document recovery time.

Validation

- Encryption key and volume tags are correct.
- Modification reaches completed/optimized state and guest capacity is expanded.
- Restored checksums and application validation succeed.
- Measured restore time and first-read performance are recorded.
- No credential or sensitive data appears in user data or tags.

Cleanup

- Unmount and delete restored/test volumes only after verifying they are not the root device.
- Delete or retain the snapshot according to the documented lab retention decision.

Interview Questions

1. Why is an EBS volume Availability-Zone scoped?
2. Compare gp3 and io2.

3. How do IOPS, throughput, and I/O size relate?
4. Why can more provisioned IOPS fail to improve performance?
5. What is the difference between crash-consistent and application-consistent snapshots?
6. What must be done after increasing volume size?
7. When is Fast Snapshot Restore useful?
8. Why is Multi-Attach dangerous for a normal filesystem?

Chapter Summary

- EBS is durable AZ-scoped block storage whose behavior includes the guest and instance limits.
- Volume selection is a performance-and-recovery decision, not just a capacity choice.
- Encryption, KMS, snapshots, and restore validation form one control system.
- Multi-Attach requires cluster-aware coordination and fencing.

Official AWS References

EBS volume types - <https://docs.aws.amazon.com/ebs/latest/userguide/ebs-volume-types.html>

EBS encryption - <https://docs.aws.amazon.com/ebs/latest/userguide/ebs-encryption.html>

EBS Multi-Attach - <https://docs.aws.amazon.com/ebs/latest/userguide/ebs-volumes-multi.html>

Amazon EFS and Amazon FSx

Managed shared file systems for Linux, Windows, HPC, enterprise NAS, and specialized data workloads

VOLUME 6B | CHAPTER 18

Learning Objectives

- Distinguish shared file storage from block and object storage.
- Design EFS mount targets, security, performance, lifecycle, and access points.
- Choose among FSx for Windows File Server, Lustre, NetApp ONTAP, and OpenZFS.
- Plan hybrid identity, backup, throughput, and multi-AZ behavior.
- Troubleshoot DNS, network, permissions, and workload mismatches.

18.1 File semantics and service selection

A file system exposes directories, names, metadata, permissions, locking, and shared access semantics. Teams often choose file storage because an application expects POSIX or SMB behavior, not because it is automatically the cheapest or fastest choice. Shared files introduce concurrency, namespace, identity, mount, and metadata considerations that block or object designs may avoid.

Amazon EFS provides managed NFS file storage designed for elastic Linux-oriented workloads. Amazon FSx provides managed file systems based on purpose-built technologies: Windows File Server for SMB and Microsoft integration, Lustre for high-performance parallel workloads, NetApp ONTAP for enterprise NAS features, and OpenZFS for ZFS-based file workloads.

18.2 Amazon EFS architecture and security

EFS is regional, and clients connect through mount targets created in subnets. A resilient design normally provides a mount target in each Availability Zone that contains clients, with DNS directing clients to the appropriate endpoint. Security groups control NFS network reachability, while POSIX ownership/mode bits and optional IAM authorization control file access.

EFS access points present an application-specific entry path and identity, reducing the need for every container or service to coordinate a shared root. Encryption in transit uses the EFS mount helper and TLS. Encryption at rest is selected at creation and governed by the chosen KMS key. Root access and anonymous identity behavior should be deliberately restricted.

18.3 EFS performance, throughput, and lifecycle

General Purpose performance mode is appropriate for most latency-sensitive workloads. Max I/O exists for specialized highly parallel cases with different latency characteristics and should not be chosen casually. Throughput modes allow elastic or provisioned behavior according to service capabilities; selection must be based on active data, operation mix, and sustained/peak demand.

Lifecycle management can transition eligible files to lower-cost classes and restore them on access, but applications must tolerate first-access latency. EFS Intelligent-Tiering and replication or backup options should be assessed against access patterns, RPO, RTO, and Region strategy. Metadata-heavy workloads with many small files require realistic tests.

18.4 FSx portfolio

FSx for Windows File Server supplies managed SMB, Active Directory integration, Windows ACLs, shadow copies, and multi-AZ deployment options for Windows applications and user shares. Identity, DNS, directory reachability, and Kerberos time synchronization are operational dependencies.

FSx for Lustre provides a parallel file system for HPC, machine learning, media processing, and analytics and can integrate with S3 data repositories. It emphasizes throughput and parallelism rather than general enterprise NAS semantics.

FSx for NetApp ONTAP offers multiprotocol access and enterprise storage features such as snapshots, cloning, tiering, and replication concepts familiar to ONTAP users. FSx for OpenZFS supports NFS and ZFS capabilities such as snapshots and clones. Service edition, deployment type, throughput capacity, storage class, and protocol support must be verified for the selected Region.

18.5 Migration, backup, and operations

AWS DataSync can move data between on-premises storage, AWS file services, and S3 with verification and scheduling. Migration design must preserve required ownership, permissions, links, timestamps, sparse files, and application consistency. Cutover includes change freeze, final synchronization, DNS or mount changes, validation, and rollback.

Backup is separate from multi-AZ availability. Use supported native snapshots, AWS Backup, replication, or application-aware mechanisms according to the service. Test restoration of data plus identity, DNS, mount configuration, security groups, and application behavior.

Managed file service selection

Service	Best fit	Key dependencies
EFS	Elastic NFS for Linux apps, containers, shared web content	Mount targets, NFS SG, POSIX/IAM, metadata pattern
FSx for Windows File Server	SMB, Windows applications, user shares	Active Directory, DNS, Kerberos, Windows ACLs
FSx for Lustre	Parallel HPC/ML/analytics throughput	Client support, data repository, workload parallelism
FSx for NetApp ONTAP	Enterprise NAS, multiprotocol, ONTAP features	Storage VM/volume design, protocol identity, tiering
FSx for OpenZFS	NFS workloads needing ZFS features	Throughput/storage configuration and client behavior

Enterprise Architecture

Shared file platform

Applications in multiple AZs -> local EFS mount targets or selected FSx endpoints

V

Security groups and private DNS -> controlled network access

V

POSIX, IAM, Active Directory, or protocol identities -> file authorization

V

Lifecycle, snapshots, backup, replication, and DataSync -> data protection/migration

V

CloudWatch, client logs, audit evidence, throughput/capacity alarms -> operations

EFS and FSx inspection

```
aws efs describe-file-systems
aws efs describe-mount-targets --file-system-id fs-EXAMPLE
aws efs describe-access-points --file-system-id fs-EXAMPLE
aws fsx describe-file-systems

# Linux client evidence
mount | grep -E 'efs|nfs'
nfsstat -m
```

Production Best Practices

- Create EFS mount targets in every client AZ required by the availability design.
- Use access points and application-specific identities instead of a universally writable root.
- Encrypt at rest and in transit and govern the KMS key lifecycle.
- Load-test metadata operations and small-file workloads, not just sequential throughput.
- Choose FSx technology from protocol and application semantics before feature familiarity.
- For Windows, treat AD, DNS, time, and network reachability as part of the file service.
- Use DataSync reports and application validation for migration, not file count alone.
- Test backup restoration including permissions and mount dependencies.

Troubleshooting

Symptom	Likely cause	Evidence / action
EFS mount times out	Missing mount target, route, NFS SG, or NACL	Resolve DNS, identify AZ target, test TCP 2049, inspect flow logs

Symptom	Likely cause	Evidence / action
Mount denied	POSIX/IAM/access-point or root-squash policy	Inspect mount options, caller role, access-point identity, ownership, and policy
Slow small-file workload	Metadata-intensive pattern or client concurrency	Measure operation mix, client options, throughput mode, and directory layout
Windows share authentication fails	AD/DNS/Kerberos time or trust issue	Validate domain controller reachability, DNS suffix, time, SPN, and ACLs
Lustre underperforms	Insufficient parallelism or mismatched client/data layout	Measure stripe/worker behavior, client network, and storage configuration
Lifecycle surprise	File moved to lower-cost class and first access is slower	Inspect lifecycle policy and access pattern; stage hot data when required
Migration counts match but app fails	Ownership, ACL, links, case, locks, or application consistency lost	Compare metadata and perform application-level validation/rollback

Hands-on Lab: Secure multi-AZ EFS application share

Objective: Create encrypted EFS with mount targets, an access point, TLS mounts, lifecycle policy, and a tested backup/restore path.

Implementation Tasks

1. Create an encrypted test file system with tags and a governed key.
2. Create mount targets in two private subnets and an EFS client security group.
3. Create an access point with a dedicated path and POSIX identity.
4. Mount through the EFS helper with TLS from two Session Manager instances.
5. Write concurrent test data and verify permissions and visibility.
6. Configure a lab lifecycle policy and capture performance/operation metrics.
7. Create a supported backup and restore to a separate validation location.

Validation

- Clients use same-AZ mount-target resolution and TCP 2049 only from approved SGs.
- Access-point identity prevents unintended root access.
- TLS mount and encryption-at-rest settings are verified.
- Concurrent data and restored metadata pass checks.
- Measured first access and normal operation are recorded.

Cleanup

- Unmount clients and delete restored/test data, access point, mount targets, filesystem, backup recovery points, and test key only when safe.
- Verify security groups and log groups are not orphaned.

Interview Questions

1. How is EFS different from EBS?
2. Why create an EFS mount target in each client AZ?
3. What problem does an EFS access point solve?
4. How do EFS throughput and performance modes differ conceptually?
5. When would you choose FSx for Windows rather than EFS?
6. What workload benefits from FSx for Lustre?
7. What dependencies commonly break SMB authentication?
8. Why is a successful file copy not sufficient migration validation?

Chapter Summary

- File storage adds namespace, concurrency, identity, mount, and metadata semantics.
- EFS is managed elastic NFS; FSx offers purpose-built Windows, Lustre, ONTAP, and OpenZFS systems.
- Performance and availability depend on client placement, protocol, operation mix, and service configuration.
- Migration and recovery must preserve permissions and application behavior, not only bytes.

Official AWS References

Amazon EFS performance - <https://docs.aws.amazon.com/efs/latest/ug/performance.html>

EFS mount targets - <https://docs.aws.amazon.com/efs/latest/ug/accessing-fs.html>

Amazon FSx documentation - <https://docs.aws.amazon.com/fsx/>

Amazon S3 Masterclass

Secure object storage, lifecycle economics, versioning, replication, immutability, events, and large-scale operations

VOLUME 6B | CHAPTER 19

Learning Objectives

- Design S3 buckets around object semantics and ownership boundaries.
- Apply Block Public Access, least privilege, encryption, and policy evaluation.
- Choose storage classes and lifecycle transitions from access and recovery data.
- Use versioning, replication, Object Lock, and events safely.
- Operate high-scale transfers, inventories, batch actions, and recovery.

19.1 Object storage mental model

Amazon S3 stores objects in buckets. An object has a key, data, metadata, and optional version identity; S3 does not expose a mounted block device or traditional filesystem transactions. Prefixes organize names but are not independent directories. Applications should embrace object APIs, immutable writes, idempotency, and explicit metadata rather than emulate shared-file assumptions.

S3 provides strong read-after-write consistency for object PUT/DELETE and metadata operations within its documented model. This does not create multi-object transactions, prevent logical overwrite, or make downstream caches and applications instantly consistent. Unique keys and manifest-based publishing simplify reliable pipelines.

19.2 Security, ownership, and encryption

S3 Block Public Access should normally be enabled at organization, account, and bucket levels unless a reviewed public use case exists. Object Ownership with bucket-owner-enforced settings disables ACLs for most modern designs and centralizes authorization in IAM, bucket policies, access points, service control policies, VPC endpoint policies, and KMS controls.

Server-side encryption protects data at rest. SSE-S3 uses S3-managed keys, SSE-KMS integrates governed KMS keys and audit controls, and dual-layer options may apply to specific compliance needs. KMS request rates, permissions, key availability, cross-account grants, and cost are part of the architecture. TLS, signature validation, and condition keys such as secure transport and organization/account constraints protect data in transit and access context.

19.3 Storage classes and lifecycle

S3 Standard supports frequently accessed data. Intelligent-Tiering can automatically move eligible objects among access tiers based on observed access. Standard-IA and One Zone-IA trade storage price for retrieval and minimum-duration considerations; One Zone-IA is not resilient to loss of an Availability Zone. Glacier Instant Retrieval, Flexible Retrieval, and Deep Archive address increasingly infrequent access and differing restoration expectations.

Lifecycle rules transition or expire current and noncurrent versions according to prefix, tags, size, age, and supported constraints. A transition that saves storage may increase request, retrieval, early-deletion, monitoring, or operational cost. Model total cost and recovery time before applying broad rules. Use S3 Storage Lens, Inventory, access data, and billing evidence to refine policy.

19.4 Versioning, replication, and immutability

Versioning preserves multiple versions and introduces delete markers instead of permanently erasing an object through a normal versioned delete. It improves recovery from overwrite and deletion but also increases retained storage; noncurrent-version lifecycle and MFA/privileged deletion governance require design.

Same-Region and Cross-Region Replication asynchronously copy eligible objects according to rules and permissions. Replication is useful for ownership, compliance, latency, and recovery patterns but is not a substitute for independent backup because harmful changes may replicate. Existing objects require Batch Replication or another migration process. Replication Time Control provides a documented service target for eligible configurations when required.

S3 Object Lock uses retention periods and legal holds on versioned buckets to provide write-once-read-many controls. Governance mode permits privileged bypass; compliance mode is intentionally more restrictive. Retention must be tested with legal, security, data-lifecycle, and KMS-key policies because an unreadable encrypted immutable object is not a successful archive.

19.5 Events and large-scale operations

S3 event notifications can publish object events to supported destinations such as EventBridge, SNS, SQS, or Lambda. Delivery is at least once and can be out of order, so consumers need idempotency, version identity, duplicate handling, poison-message management, and replay strategy. Avoid event loops where a consumer writes to the triggering prefix without filtering.

Multipart upload improves large-object transfer reliability and parallelism. Incomplete multipart uploads consume storage until completed or aborted. S3 Inventory produces object metadata reports, Batch Operations applies actions at scale, and Storage Lens provides organization-level visibility. Checksums and application-level validation should be part of ingest and restore.

S3 protection and economics matrix

Control	Protects against	Does not automatically solve
Block Public Access	Accidental public policies/ACLs	Private over-permission or stolen identities
Versioning	Overwrite and normal delete recovery	Retention cost, compromised privileged deletion
Replication	Regional/account copy objectives	Independent point-in-time backup by itself
Object Lock	Deletion/overwrite during retention	Key loss, wrong retention, application corruption before write

Control	Protects against	Does not automatically solve
Lifecycle	Automated class transition/expiration	Correct classification without measurement
Event notification	Automated reaction to object events	Exactly-once ordered processing

Enterprise Architecture

Governed data lake or object platform

Producers -> private endpoints/TLS -> dedicated buckets or access points

V

Bucket-owner-enforced ownership, Block Public Access, IAM/bucket/SCP/KMS policy

V

Versioning, checksums, lifecycle, replication, and Object Lock by data class

V

Events -> SQS/EventBridge -> idempotent processors with dead-letter/replay

V

Inventory, Storage Lens, CloudTrail data events, access logs, Config, and recovery tests

S3 control inspection

```
aws s3api get-public-access-block --bucket BUCKET
aws s3api get-bucket-ownership-controls --bucket BUCKET
aws s3api get-bucket-encryption --bucket BUCKET
aws s3api get-bucket-versioning --bucket BUCKET
aws s3api get-bucket-lifecycle-configuration --bucket BUCKET
aws s3api list-object-versions --bucket BUCKET --prefix test/
```

Production Best Practices

- Enable organization/account/bucket Block Public Access unless public access is explicitly governed.
- Prefer bucket-owner-enforced ownership and policy-based access over ACLs.
- Separate data by ownership, sensitivity, retention, and blast radius rather than one universal bucket.
- Enforce TLS and tightly govern cross-account, VPC endpoint, and KMS conditions.
- Enable versioning before relying on replication or Object Lock and manage noncurrent versions.
- Model retrieval, request, minimum-duration, monitoring, and transition costs.
- Make event consumers idempotent and retain replay evidence.
- Abort incomplete multipart uploads and inventory large estates.
- Regularly restore selected versions and verify checksums, metadata, permissions, and application use.

Troubleshooting

Symptom	Likely cause	Evidence / action
AccessDenied	IAM/bucket/SCP/endpoint/KMS explicit deny or ownership issue	Identify caller and request context; inspect all policy layers and CloudTrail
Object appears missing	Wrong Region/account/key/version or delete marker	Use head/list versions, exact key encoding, bucket owner, and endpoint
Public site stopped	Block Public Access, ownership, policy, or CloudFront origin control changed	Confirm approved architecture; do not disable guardrails broadly
Replication pending/failed	Role, KMS, ownership, versioning, destination, or rule filter	Inspect replication status, metrics, permissions, and failed-object reports
Lifecycle did not transition	Rule filter, object size/class, timing, or unsupported sequence	Inspect effective rule, object metadata, class constraints, and age
Event processed twice	At-least-once delivery	Use event/object version idempotency and deduplication
KMS throttling or cost spike	High request rate or bucket-key/configuration pattern	Inspect KMS metrics, data-key pattern, permissions, and architecture
Archive cannot restore in time	Storage class or retrieval tier mismatches RTO	Initiate tested retrieval and revise class/recovery design

Hands-on Lab: Secure versioned S3 data workflow

Objective: Build a private encrypted bucket with versioning, lifecycle, event processing, and a tested recovery of deleted and overwritten data.

Implementation Tasks

1. Create a uniquely named bucket with Block Public Access and bucket-owner-enforced ownership.
2. Configure default encryption and a policy that denies insecure transport.
3. Enable versioning and upload several versions with checksums and tags.
4. Configure a scoped lifecycle rule for noncurrent versions and incomplete multipart uploads.
5. Publish test object events to SQS or EventBridge and implement idempotent observation.
6. Delete and overwrite test objects, then recover selected versions.
7. Optionally configure a separate destination bucket and scoped replication rule in a sandbox.

Validation

- No public access path exists.
- Caller, bucket, and KMS policies allow only the intended operation.
- Version IDs and delete markers are visible and recoverable.
- Lifecycle and event configurations match the scoped prefix/tags.
- Duplicate event handling does not duplicate business processing.

- Recovered checksum and metadata match the chosen version.

Cleanup

- Remove event destinations, replication rules, test objects and all versions/delete markers before deleting buckets.
- Retain immutable or recovery objects only according to the documented policy; do not attempt unsafe bypass.

Interview Questions

1. How is an S3 prefix different from a directory?
2. What does bucket-owner-enforced Object Ownership change?
3. Why can AccessDenied require checking KMS and endpoint policy?
4. Versioning versus replication: what does each protect?
5. Why is replication not automatically a backup?
6. What are Object Lock governance and compliance modes?
7. Why must S3 event consumers be idempotent?
8. How would you choose a storage class?

Chapter Summary

- S3 is object storage with API, key, metadata, and version semantics.
- Security requires aligned identity, bucket, organization, endpoint, ownership, and KMS controls.
- Lifecycle economics and recovery time must be measured across the full object lifecycle.
- Versioning, replication, Object Lock, and events solve different problems and require tested operations.

Official AWS References

S3 storage classes - <https://docs.aws.amazon.com/AmazonS3/latest/userguide/storage-class-intro.html>

S3 Versioning - <https://docs.aws.amazon.com/AmazonS3/latest/userguide/Versioning.html>

S3 Object Lock - <https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lock.html>

S3 Event Notifications - <https://docs.aws.amazon.com/AmazonS3/latest/userguide/EventNotifications.html>

Enterprise Storage Architecture, Backup, and Disaster Recovery

Workload-driven service selection, RPO/RTO engineering, policy-based backup, recovery testing, and the Part B capstone

VOLUME 6B | CHAPTER 20

Learning Objectives

- Select EBS, EFS, FSx, S3, or ephemeral storage from application semantics.
- Translate business impact into RPO, RTO, retention, and recovery tiers.
- Design AWS Backup, snapshots, replication, vault, account, and Region controls.
- Distinguish high availability, replication, backup, archive, and disaster recovery.
- Execute a capstone architecture and evidence-based recovery game day.

20.1 Start with data behavior, not product names

Storage architecture begins with access semantics: block, shared file, object, database, stream, cache, or scratch. Then quantify capacity growth, object/file count, I/O size, concurrency, latency, throughput, durability, consistency, retention, sovereignty, encryption, sharing, and recovery. No single service is the correct default for every data type.

Use EBS when an EC2 workload needs an attached block device in one Availability Zone. Use EFS for managed elastic NFS sharing, an FSx technology for its specific protocol and feature set, and S3 for object-native durable storage and broad lifecycle options. Instance store is for disposable local data. Databases should normally use purpose-built managed data services when their transaction and operation model requires them.

20.2 Availability, durability, backup, and DR are different

High availability keeps a service operating through expected component failure. Durability is the probability that stored data remains intact. Backup creates separately managed recovery points. Replication copies data to another location but can copy corruption or deletion. Archive optimizes long-term retention and retrieval economics. Disaster recovery restores business service after a larger failure.

A complete recovery includes compute definitions, networking, DNS, identity, secrets, keys, certificates, data, application configuration, observability, runbooks, and people. A data copy without a deployable application or usable KMS key does not meet the business RTO.

20.3 RPO, RTO, and recovery tiers

Recovery point objective is the maximum acceptable data loss measured in time. Recovery time objective is the target time to restore the service. These are business decisions translated into engineering controls. They must include dependencies and be validated with measured exercises.

Backup-and-restore is cost efficient but has a longer RTO. Pilot light keeps critical core components ready. Warm standby runs a scaled-down functional environment. Multi-site active-active offers the shortest potential interruption but has the greatest application, data-consistency, operational, and cost complexity. Choose per workload tier rather than declaring one enterprise-wide DR pattern.

20.4 AWS Backup and protection boundaries

AWS Backup centralizes policy-based protection for supported services using plans, selections, vaults, lifecycle, copy actions, reporting, and restore testing capabilities. Organizations can apply policies across accounts. Vault Lock and logically isolated vault patterns can strengthen resistance to malicious or accidental deletion when governed correctly.

A robust model separates workload administration from backup administration, copies recovery points to a dedicated backup account and another Region when required, restricts KMS and vault deletion, monitors failed jobs, and periodically restores. Retention settings, legal holds, service-native features, and AWS Backup behavior must be reconciled to avoid gaps or unexpected cost.

20.5 Recovery engineering and game days

A recovery runbook must state trigger authority, prerequisites, dependency order, automation, validation, communication, security checks, data-loss window, and rollback/fail-forward decision. Restore into an isolated validation environment before production when possible. Validate identity, decryption, DNS, data integrity, application transactions, observability, capacity, and security posture.

Game days deliberately inject a realistic failure such as an Availability Zone loss, accidental object deletion, corrupted deployment, unavailable KMS permission, or primary-Region outage. Observers record detection time, decision time, restoration time, actual data loss, manual steps, bottlenecks, and unsafe assumptions. Findings become owned remediation work.

Storage and recovery selection

Requirement	Primary pattern	Evidence to collect
Boot or single-host block device	Encrypted EBS plus tested snapshots	Latency, IOPS/throughput, filesystem and restore
Shared Linux namespace	EFS with mount targets/access points	Metadata operations, permissions, mount resilience
SMB or specialized file semantics	Appropriate FSx service	Identity, protocol, throughput, failover and backup
Object-native durable data	S3 with version/lifecycle/replication as required	Policy, versions, checksums, retrieval and restore
Disposable high-speed scratch	Instance store or ephemeral service layer	Reconstruction path and data-loss acceptance
Cross-account/Region recovery	Policy-based copies plus deployable recovery environment	RPO/RTO game-day measurement and key access

Enterprise Architecture

Enterprise recovery control plane

Workload accounts -> encrypted EBS/EFS/FSx/S3 and application-native protection

V

AWS Organizations backup policies -> tagged resource selection and schedules

V

Dedicated backup account -> restricted vaults, independent KMS administration, audit logs

V

Secondary Region/account -> copied recovery points and deployable infrastructure definitions

V

Restore orchestration -> isolated validation -> DNS/capacity/security approval -> service recovery

V

Audit Manager reports, job alarms, restore test results, game-day actions, and executive metrics

Backup and recovery inspection

```
aws backup list-backup-vaults
aws backup list-backup-plans
aws backup list-backup-jobs
aws backup list-restore-jobs
aws backup list-recovery-points-by-backup-vault --backup-vault-name VAULT

# Record actual start/completion times and business validation separately.
```

Production Best Practices

- Classify data and assign owner, criticality, RPO, RTO, retention, legal, and sovereignty requirements.
- Use separate backup administration and destination accounts for critical workloads.
- Copy recovery points across Regions only where the threat model and business objective justify it.
- Protect keys, vaults, policies, and audit trails from the same identity that administers workloads.
- Monitor backup selection drift as well as job failure.
- Test restores automatically where possible and perform full application game days.
- Measure actual RPO/RTO including DNS, identity, secrets, capacity, and validation.
- Delete obsolete copies only through approved retention and legal processes.
- Treat runbooks and Infrastructure as Code as recovery assets and back them up independently.

Troubleshooting

Symptom	Likely cause	Evidence / action
Backup job never started	Resource not selected, unsupported config, role, tag, or policy issue	Inspect effective backup plan/selection, service support, IAM, and organization policy
Copy job failed	Destination vault/key policy, Region, service, or permission issue	Inspect job status message, vault access, KMS grants, and CloudTrail
Restore completes but service fails	Network, identity, secrets, DNS, application config, or dependency missing	Validate full dependency graph and deployment artifacts
RTO exceeded	Manual steps, slow retrieval, capacity shortage, or untested sequence	Review timeline, automate bottlenecks, pre-provision required control plane
RPO exceeded	Schedule/frequency or replication lag did not meet objective	Compare latest usable point with incident time and redesign frequency
Recovery point cannot be deleted	Retention, Vault Lock, legal hold, or permission protection	Follow governance process; never weaken controls as an ad-hoc fix
Backup costs spike	Unbounded retention, copies, changed data, or unused recovery points	Analyze inventory by owner/tier and apply approved lifecycle
Game day succeeds technically but business rejects	Validation covered infrastructure, not transactions or data quality	Define business acceptance tests and accountable approvers

Hands-on Lab: Part B capstone - resilient multi-tier service and recovery game day

Objective: Design, deploy, observe, scale, back up, and recover a two-AZ application using the core infrastructure controls from Chapters 11-20.

Implementation Tasks

1. Write requirements: users, availability, data classes, RPO, RTO, traffic, security, budget, and cleanup date.
2. Create a two-AZ VPC with public load-balancer subnets, private application subnets, isolated data subnets, zonal egress or endpoints, and flow logs.
3. Deploy a versioned launch-template/AMI application fleet behind an ALB and Auto Scaling policy.
4. Create Route 53 test DNS with an alias and documented TTL/rollback.
5. Use an encrypted storage layer appropriate to the lab: EBS data restore, EFS share, or S3 object workflow.
6. Apply least-privilege security-group references, IMDSv2, Session Manager, encryption, logs, alarms, tags, and budgets.
7. Create backup/recovery points and a separate validation restore.

8. Inject failures: terminate an instance, break a health path, remove a safe route in a controlled window, and delete/overwrite test data.
9. Recover service/data, measure detection, RPO, RTO, and business validation, then write remediation actions.

Validation

- No application instance has a public management port or stored AWS key.
- Traffic, target health, and scaling remain available through individual instance failure.
- Routes and SGs permit only documented paths; flow evidence supports the result.
- DNS alias, certificate/listener, health, logs, and rollback are verified.
- Storage performance and data recovery meet the stated lab objectives.
- Every failure and capacity change has traceable evidence.
- Actual RPO/RTO and business transaction checks are recorded.
- Costly resources and public addresses are identified before cleanup.

Cleanup

- Export architecture, commands/IaC, logs, metrics, recovery evidence, and lessons learned.
- Delete in dependency order: DNS records, load balancer/listeners/targets, ASG/instances/templates, storage/restores/backups per retention decision, endpoints/NAT/EIPs, subnets/routes/gateways/VPC, logs and temporary keys.
- Use billing and resource inventory to confirm no orphan resource remains.

Interview Questions

1. How do availability, durability, backup, replication, archive, and DR differ?
2. How do RPO and RTO change architecture and cost?
3. When would you choose EBS, EFS, FSx, or S3?
4. Why is a cross-Region replica not automatically a backup?
5. What is the difference among backup-and-restore, pilot light, warm standby, and active-active?
6. Why separate backup administration from workload administration?
7. What does a successful restore test need to validate?
8. How would you run and score a recovery game day?

Chapter Summary

- Storage selection begins with access semantics, performance, ownership, and recovery requirements.
- RPO and RTO must be business-approved and measured through full-service recovery.
- AWS Backup can centralize protection, but accounts, vaults, keys, and selection drift still require governance.
- Part B culminates in an observable, scalable, recoverable infrastructure system rather than isolated service knowledge.

Official AWS References

AWS Backup Developer Guide - <https://docs.aws.amazon.com/aws-backup/latest/devguide/whatisbackup.html>

AWS disaster recovery strategies - <https://docs.aws.amazon.com/whitepapers/latest/disaster-recovery-workloads-on-aws/disaster-recovery-options-in->

the-cloud.html

AWS Well-Architected Reliability Pillar - <https://docs.aws.amazon.com/wellarchitected/latest/reliability-pillar/welcome.html>

Appendix A - AWS CLI Command Reference

Area	Read-only command examples
Identity	aws sts get-caller-identity aws configure list
EC2	aws ec2 describe-instances aws ec2 describe-launch-templates
VPC	aws ec2 describe-vpcs aws ec2 describe-route-tables
Route 53	aws route53 list-hosted-zones
ELB	aws elbv2 describe-load-balancers aws elbv2 describe-target-health
Auto Scaling	aws autoscaling describe-auto-scaling-groups describe-scaling-activities
EBS	aws ec2 describe-volumes describe-snapshots --owner-ids self
EFS/FSx	aws efs describe-file-systems aws fsx describe-file-systems
S3	aws s3api list-buckets get-bucket-versioning list-object-versions
Backup	aws backup list-backup-jobs list-restore-jobs

Appendix B - Cross-layer Troubleshooting Matrix

Layer	Key question	Evidence
Identity	Who is the caller and is there an explicit deny?	STS, IAM evaluation, CloudTrail, KMS policy
DNS	What address and zone answered?	dig/nslookup, Route 53, Resolver logs
Routing	Which forward and return routes match?	Route tables, Transit Gateway, Reachability Analyzer
Network policy	Do SGs/NACLs permit the full flow?	Rules and VPC Flow Logs
Load balancer	Are listener, rule, and target health correct?	Target reason, access logs, metrics
Compute	Did launch and bootstrap complete?	EC2 state, cloud-init, systemd, SSM
Scaling	Which policy or health event changed capacity?	ASG activities, alarms, refresh status
Storage	Is the issue service, network, OS, key, or application?	Service metrics, mount/device output, KMS, app logs
Recovery	Can the point be decrypted and restored with dependencies?	Backup/restore jobs and business validation

Appendix C - Production Readiness Checklist

Compute

- Versioned launch template and AMI.
- IMDSv2 and least-privilege instance role.
- Deterministic bootstrap and replacement.
- Measured capacity and launch readiness.

Network and DNS

- Non-overlapping CIDRs and explicit subnet roles.
- Documented forward/return paths and egress.
- Tier-based security-group references.
- DNS delegation, TTL, health, and rollback tested.

Load Balancing and Scaling

- Multi-AZ targets and TLS.
- Health, logs, alarms, draining, and deployment rollback.
- Scaling metric, warmup, limits, and dependency ceilings tested.

Storage and Recovery

- Service matches access semantics and measured performance.
- Encryption, lifecycle, versioning, retention, and backup tested.
- RPO/RTO measured through restoration.

Operations and Cost

- Central metrics/logs/audit evidence.
- Runbooks with owner, validation, and rollback.
- Budgets, tags, expiry, and orphan cleanup.

Appendix D - Glossary

Term	Definition
ALB	Application-layer HTTP/HTTPS load balancer.
AMI	Bootable image template for EC2.
ASG	Auto Scaling group that maintains and scales a fleet.
CIDR	Notation for IP address ranges.
EBS	AZ-scoped block storage.
EFS	Managed elastic NFS file storage.
FSx	Managed purpose-built file systems.

Term	Definition
GWLB	Load balancer for virtual network appliances.
IMDSv2	Token-based EC2 metadata service.
IOPS	Input/output operations per second.
NACL	Stateless subnet-level access list.
NLB	Transport-layer load balancer.
RPO	Maximum acceptable data loss measured in time.
RTO	Target time to restore service.
TTL	DNS cache lifetime.
VPC	Regional virtual network boundary.

Appendix E - Official Documentation Families

- Amazon EC2 User Guide - <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/>
- Amazon EC2 Auto Scaling User Guide - <https://docs.aws.amazon.com/autoscaling/ec2/userguide/>
- Amazon VPC User Guide - <https://docs.aws.amazon.com/vpc/latest/userguide/>
- Amazon Route 53 Developer Guide - <https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/>
- Elastic Load Balancing User Guide - <https://docs.aws.amazon.com/elasticloadbalancing/latest/userguide/>
- Amazon EBS User Guide - <https://docs.aws.amazon.com/ebs/latest/userguide/>
- Amazon EFS User Guide - <https://docs.aws.amazon.com/efs/latest/ug/>
- Amazon FSx Documentation - <https://docs.aws.amazon.com/fsx/>
- Amazon S3 User Guide - <https://docs.aws.amazon.com/AmazonS3/latest/userguide/>
- AWS Backup Documentation - <https://docs.aws.amazon.com/aws-backup/>

End of Volume 6B

You have completed AWS Core Infrastructure. The next part continues with database and data-platform engineering, building on the compute, network, traffic, scaling, storage, and recovery controls established here.